

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №1
З дисципліни "Генеративні нейронні мережі та їх застосування"
Використання та fine-tuning великих мовних моделей

Виконав:
студент гр. ІПЗм-25-2
Вовк Дмитро Андрійович

Перевірила: доцент кафедри ІІ
Шевченко Олена Леонідівна

Харків 2026

Мета: вміти розгортати та використовувати попередньо навчені LLM, проводити їх донавчання (fine-tuning) для виконання специфічних задач за допомогою бібліотеки LoRA та оцінювати якість отриманої донавченої моделі.

Хід роботи:

Видалимо з шаблону ноутбуку навчання та тестування роботи моделі на наборі даних leprechaun та залишимо лише роботу з yoda стилем. Для цього видалимо всі блоки з (style= «leprechaun»)

Задамо початковий seed наступним, для відтворюваності результатів, чином:

```
[3]
✓ 21 c

import os
import json
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt

import torch
from torch.nn import functional as F
from torch.utils.data import DataLoader

from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from lion_pytorch import Lion
from itertools import islice

<frozen importlib._bootstrap>:488: DeprecationWarning: builti
<frozen importlib._bootstrap>:488: DeprecationWarning: builti
/usr/local/lib/python3.12/dist-packages/jupyter_client/session
return datetime.utcnow().replace(tzinfo=utc)

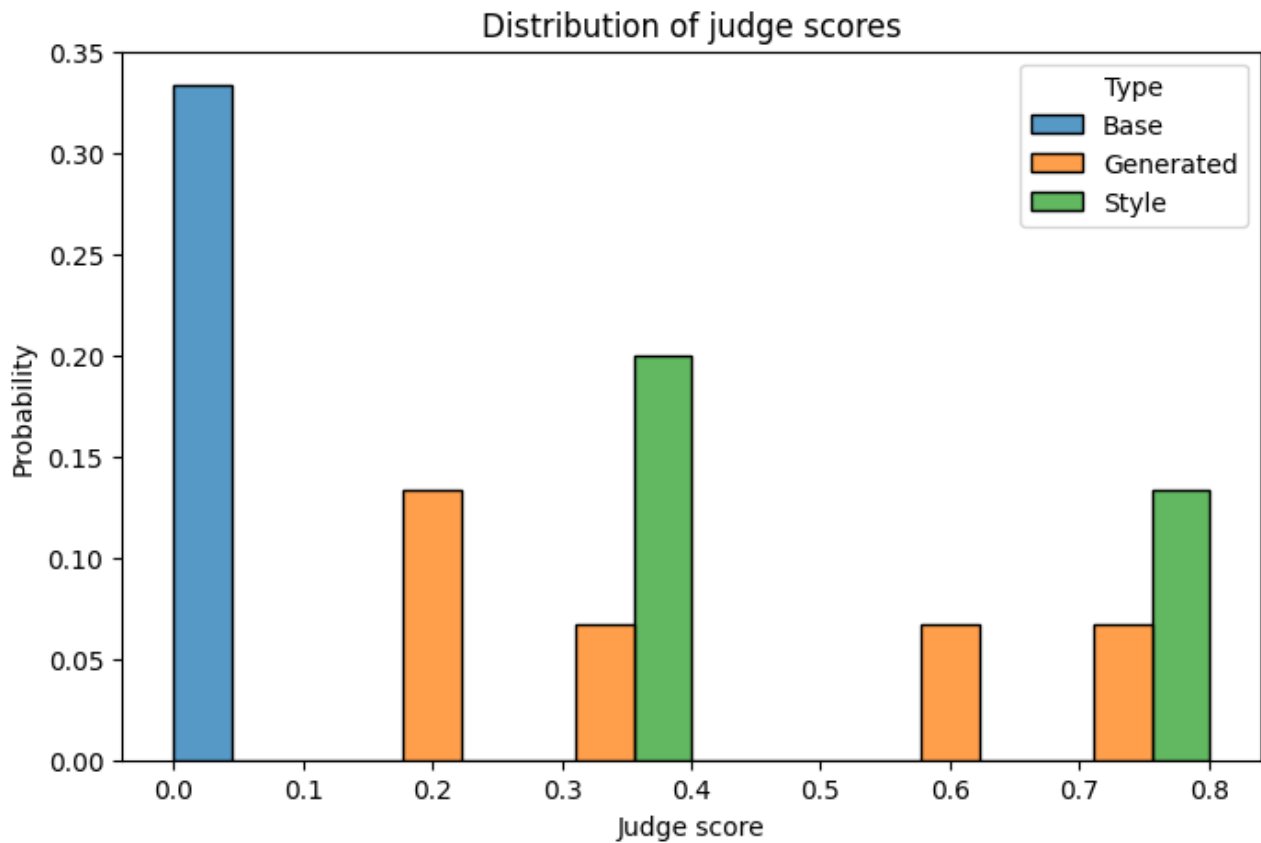
[4]
✓ 0 c

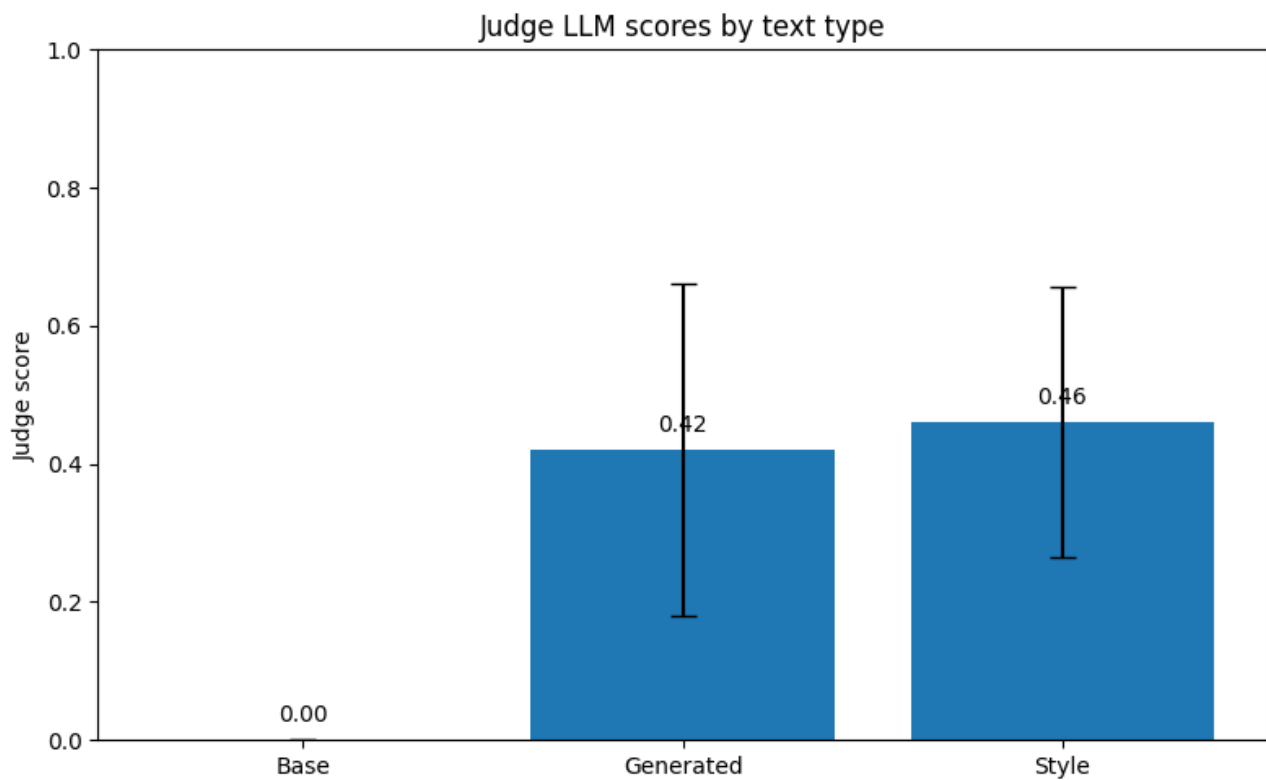
from transformers import set_seed
import random
SEED = 995 # your seed value
set_seed(SEED)
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

```
# Load the Yoda-speak dataset and fine-tune the model using your training function
set_seed(SEED)
train_loader, test_loader = mdl.lab3.create_dataloader(style="yoda")
model = train(model, train_loader, tokenizer, max_steps=1600, learning_rate=5e-5, run_name="yoda")
```

```
set_seed(SEED)
judge = LLMJudgeEvaluator(llm, system_prompt=system_prompt)
```

Повторимо експеримент 2–3 рази з різними значеннями seed і усереднимо отримані оцінки якості моделі. Спочатку запусимо всі блоки із seed = 995. Після отримання samples оцінимо їх за допомогою LLM-судді.





```
# DO NOT CHANGE/MODIFY THIS CELL.
# EXECUTE IT BEFORE SUBMITTING YOUR ENTRY TO THE LAB.

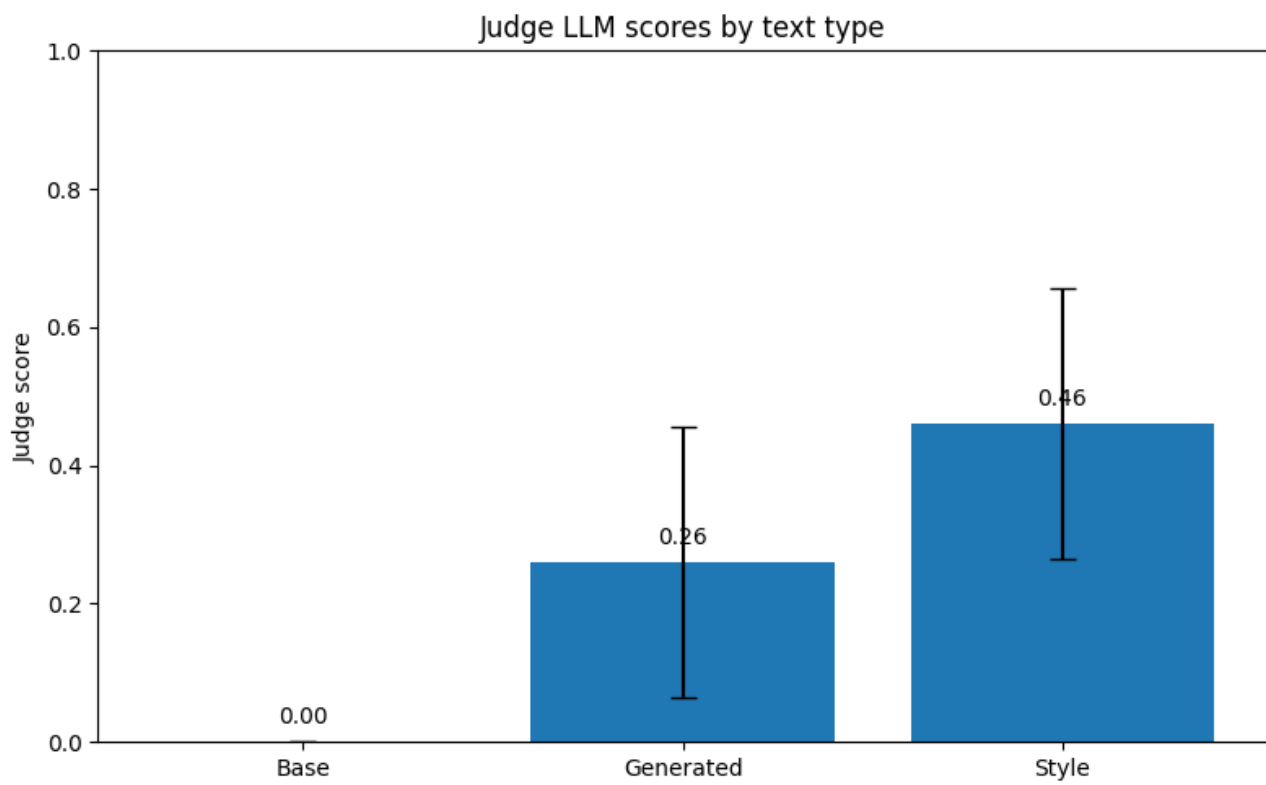
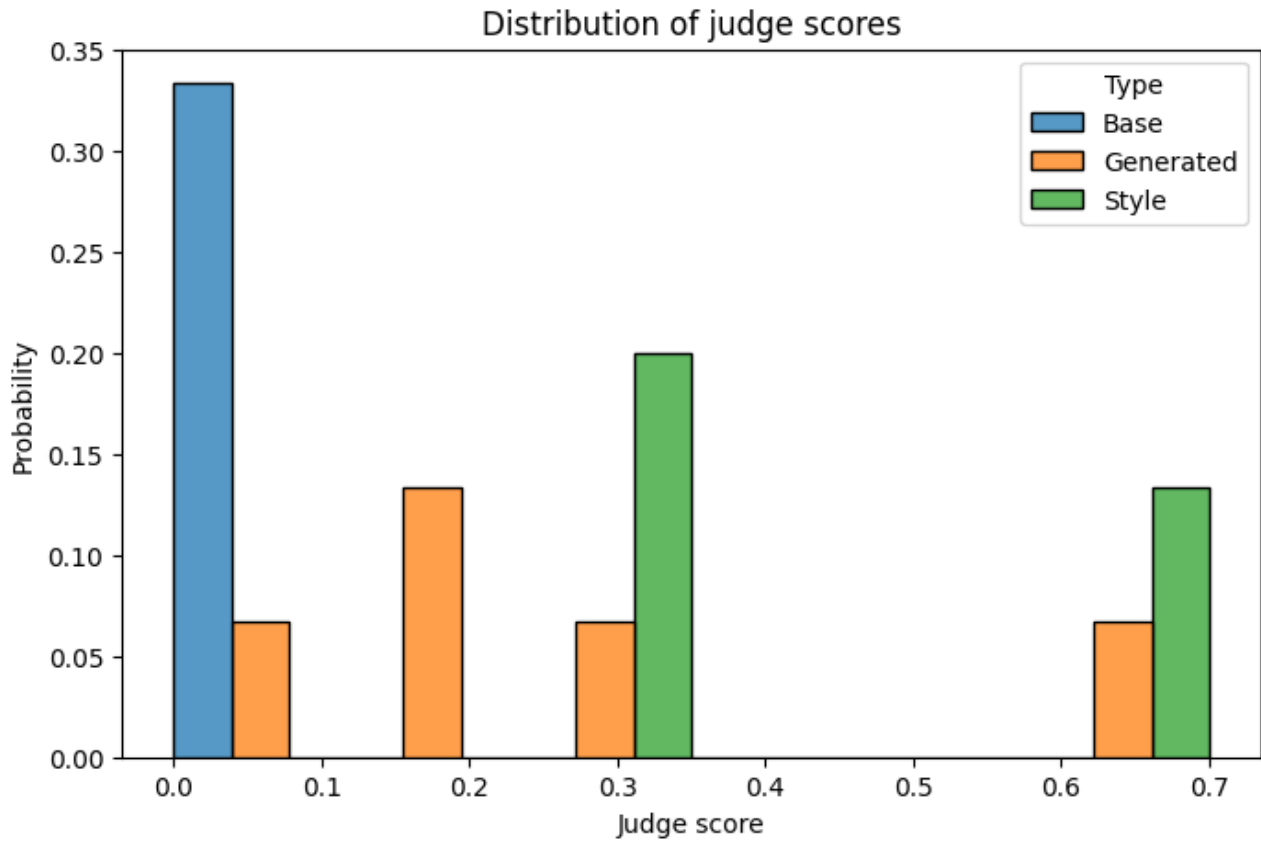
yoda_test_text = mdl.lab3.yoda_test_text
tokens = tokenizer(yoda_test_text, return_tensors="pt").to(model.device)

# Get the loglikelihood from the model
with torch.no_grad():
    outputs = model(**tokens)
    logits = outputs.logits[:, :-1]
    targets = tokens.input_ids[:, 1:]
    loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                           targets.reshape(-1))

print(f"Yoda test loglikelihood: {loss.item():.2f}")

... Yoda test loglikelihood: 5.09
```

Тепер запусимо всі блоки із сідом 999



```
# DO NOT CHANGE/MODIFY THIS CELL.
# EXECUTE IT BEFORE SUBMITTING YOUR ENTRY TO THE LAB.

yoda_test_text = mdl.lab3.yoda_test_text
tokens = tokenizer(yoda_test_text, return_tensors="pt").to(model.device)

# Get the loglikelihood from the model
with torch.no_grad():
    outputs = model(**tokens)
    logits = outputs.logits[:, :-1]
    targets = tokens.input_ids[:, 1:]
    loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                           targets.reshape(-1))

print(f"Yoda test loglikelihood: {loss.item():.2f}")

... Yoda test loglikelihood: 3.06
```

За підсумками першого запуску значення LogLikelihood дорівнювало 5.09, а другого - 3.06. Середнє значення метрики становить 4.075, що вказує на стабільність моделі та невеликі відмінності між окремими запусками.

Оцінювання за допомогою Judge LLM проводилось для трьох типів текстів: Base, Generated і Style. Контрольні тексти типу Base в обох випадках отримали оцінку 0.00. Для текстів типу Generated значення склали 0.42 і 0.26 (у середньому - 0.34). Для текстів типу Style в обох запусках отримано однакову оцінку - 0.46.

Результати свідчать, що модель працює стабільно: різниця між значеннями LogLikelihood у різних запусках є помірною, а середнє значення підтверджує відсутність значних коливань якості.

Оцінювання Judge LLM показало, що базові (контрольні) тексти коректно розпізнаються як еталонні (0.00), що підтверджує адекватність метрики. Для згенерованих текстів спостерігається середній рівень якості (0.34), що вказує на наявність простору для покращення. Водночас тексти зі стилізацією демонструють стабільно вищий результат (0.46), що свідчить про здатність моделі краще відтворювати заданий стиль, ніж генерувати загальний зміст.

Загалом модель є стабільною, але потребує покращення якості генерації звичайних (Generated) текстів.

Далі виконаємо завдання на оцінку **добре**:

Додамо метрику StyleGain, щоб кількісно виміряти покращення стилю після fine-tuning. Вона обчислюється як різниця між середніми оцінками судді для текстів, згенерованих моделлю після навчання, і базовими текстами до навчання.

Код:

```
def compute_style_metrics(style_scores, base_scores):
    P = np.array(style_scores)
    N = np.array(base_scores)

    recall = np.mean(P)
    specificity = np.mean(1 - N)
    balanced_acc = 0.5 * (recall + specificity)
    style_gain = np.mean(P) - np.mean(N)

    return {
        "Recall": recall,
        "BalancedAccuracy": balanced_acc,
        "StyleGain": style_gain
    }

metrics = compute_style_metrics(style_scores, base_scores)
print("Metrics:", metrics)
```

Результати виконання:

```
Metrics: {'Recall': np.float64(0.43999999999999995), 'BalancedAccuracy': np.float64(0.72), 'StyleGain': np.float64(0.43999999999999995)}
```

Перемішати і розділити вибірку (дивися пояснення у розділі 6) навчальних прикладів на train, validation і test, задати batch_size > 10. Порівняти результати з попередніми на основі обраних метрик.

Код:

```
from datasets import load_dataset
from torch.utils.data import DataLoader
import random

def load_yoda_style(path="content/yoda.txt"):
    with open(path, "r", encoding="utf-8") as f:
        lines = [l.strip() for l in f.readlines() if len(l.strip()) > 0]
    return lines

def create_dataloaders_yoda(batch_size=16):
    # 1. Load base dataset
    dataset = load_dataset("databricks/databricks-dolly-15k", split="train")

    # 2. Load Yoda style
    yoda_lines = load_yoda_style()

    # 3. Shuffle dataset
    dataset = dataset.shuffle(seed=SEED)

    # 4. Build new dataset with style
    data = []
    for i, sample in enumerate(dataset):
        styled = yoda_lines[i % len(yoda_lines)]

        data.append({
            "instruction": sample["instruction"],
            "response": sample["response"],          # base
            "response_style": styled                  # Yoda style
        })

    # 5. Split 70/15/15
    N = len(data)
    train_end = int(0.7 * N)
    val_end = int(0.85 * N)

    train_data = data[:train_end]
    val_data = data[train_end:val_end]
    test_data = data[val_end:]

    # 6. Collate
    def collate_fn(batch):
        return {
            "instruction": [x["instruction"] for x in batch],
            "response": [x["response"] for x in batch],
            "response_style": [x["response_style"] for x in batch],
        }

    # 7. DataLoaders
    train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, collate_fn=collate_fn)
    val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False, collate_fn=collate_fn)
    test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False, collate_fn=collate_fn)

    return train_loader, val_loader, test_loader
```



```

# === використання ===
train_loader, val_loader, test_loader = create_dataloaders_yoda(batch_size=16)

# нові даталоадери
train_loader, val_loader, test_loader = create_dataloaders_yoda(batch_size=16)

# важливо: перевстановлюємо seed
set_seed(SEED)

# створюємо НОВУ модель (не використовуємо стару!)
model_new = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")

# застосовуємо LoRA
model_new = apply_lora(model_new)

# тренуємо
model_new = train(
    model_new,
    train_loader,
    tokenizer,
    max_steps=1600,
    learning_rate=5e-5,
    run_name="yoda_split"
)

generated_samples_new = generate_samples_from_test(test_loader, num_samples=5)
generated_scores_new = evaluate_samples(generated_samples_new, "generated_new")
metrics_new = compute_style_metrics(generated_scores_new, base_scores)

print("New metrics:", metrics_new)

```

Результати:

```

100%|██████████| 5/5 [00:04<00:00, 1.00it/s]
New metrics: {'Recall': np.float64(0.4), 'BalancedAccuracy': np.float64(0.7), 'StyleGain': np.float64(0.4)}

```

Додати в notebook підсумкову таблицю або графік метрик для декількох (3-4) запусків процесу навчання та тестування з різними наборами параметрів. Параметри краще обирати зі списку: learning rate (1e-4 або 2e-4 або 3e-4), lora rank (4, 8, 16, 32), context length (256, 512, 1024). Порівняння треба робити, фіксуючи усі параметри, окрім одного, його змінюємо і перевіряємо, як зміна значення параметра впливає на якість моделі. Додати висновок, яка конфігурація (за якого набору значень параметрів) краща і чому.

```

import pandas as pd
import torch

if "model_new" in locals():
    del model_new
torch.cuda.empty_cache()

results = []

configs = [
    {"lr": 1e-4, "rank": 4, "ctx": 256},
    {"lr": 1e-4, "rank": 16, "ctx": 256},
    {"lr": 1e-4, "rank": 32, "ctx": 256},
]

base_scores_global = evaluate_samples(base_samples, "base")

for cfg in configs:
    print("Running config:", cfg)

    train_loader, val_loader, test_loader = create_dataloaders_yoda(batch_size=16)

    set_seed(SEED)

    model_tmp = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")
    model_tmp = apply_lora(model_tmp, lora_rank=cfg["rank"])

    model_tmp = train(
        model_tmp,
        train_loader,
        tokenizer,
        max_steps=600,
        learning_rate=cfg["lr"],
        run_name=f"lr_{cfg['lr']}"
    )

    gen_samples = generate_samples_from_test(test_loader, num_samples=5)

    gen_scores = evaluate_samples(gen_samples, "generated_test")

    mean_score = np.mean(gen_scores)
    metrics = compute_style_metrics(gen_scores, base_scores_global)

    results.append({
        "learning_rate": cfg["lr"],
        "MeanScore": mean_score,
        "Recall": metrics["Recall"],
        "BalancedAccuracy": metrics["BalancedAccuracy"],
        "StyleGain": metrics["StyleGain"]
    })

    del model_tmp, gen_samples, gen_scores
    torch.cuda.empty_cache()

df_results = pd.DataFrame(results)
print(df_results)

```

```
plt.figure()
plt.plot(df_results["learning_rate"], df_results["BalancedAccuracy"], marker='o')
plt.xlabel("Learning rate")
plt.ylabel("BalancedAccuracy")
plt.title("LR vs Quality")
plt.show()
```

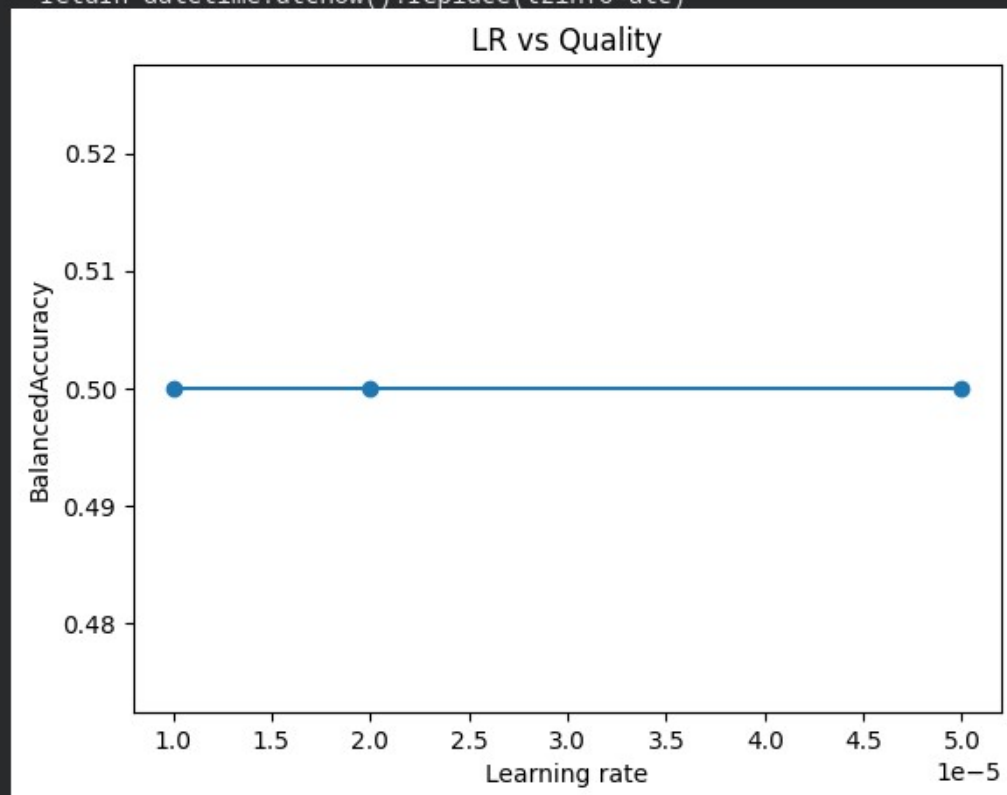
Результати:

```
df_results = pd.DataFrame(results)
print(df_results)
```

```
plt.figure()
plt.plot(df_results["learning_rate"], df_results["BalancedAccuracy"], marker='o')
plt.xlabel("Learning rate")
plt.ylabel("BalancedAccuracy")
plt.title("LR vs Quality")
plt.show()
```

	learning_rate	MeanScore	Recall	BalancedAccuracy	StyleGain
0	0.00001	0.0	0.0	0.5	0.0
1	0.00002	0.0	0.0	0.5	0.0
2	0.00005	0.0	0.0	0.5	0.0

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning:
return datetime.utcnow().replace(tzinfo=utc)
```



Результати експерименту показали, що зміна learning rate в межах $1e-5$ – $5e-5$ не вплинула на якість моделі: Balanced Accuracy залишалась 0.5, а інші метрики (MeanScore, Recall, StyleGain) дорівнювали нулю. Це також підтверджується графіками, які не демонструють жодної залежності між параметром і якістю.

Такі результати свідчать не про стабільність моделі, а радше про відсутність навчання або його неефективність. Ймовірно, learning rate занадто малий або кількість кроків навчання недостатня.

Порівняння результатів до та після Fine-Tuning

Для демонстрації ефекту Fine-Tuning візьмемо п'ять загальних тестових запитань. Спочатку отримаємо відповіді від базової моделі. Далі виконаємо донавчання моделі на датасеті з відповідями у стилі Йоди та повторно перевіримо ті самі запитання.

Програмний код:

```
questions = [
    "Tennis is?",
    "What side you chosen?",
    "Energy is?",
    "Who is Darth Vader?",
    "Explain gravity"
]

before_answers = []
for q in questions:
    ans = inference_chat(q, only_answer=True, model=old_model)
    score = scoring_function(ans)
    before_answers.append((q, ans, score))
    print(q, "|", ans, "| score:", score)

print("\n=== AFTER FT ===")
after_answers = []
for q in questions:
    ans = inference_chat(q, only_answer=True, model=model)
    score = scoring_function(ans)
    after_answers.append((q, ans, score))
    print(q, "|", ans, "| score:", score)
```

Результати:

```
*** Tennis is? | A sport, tennis is. Played on a court, with rackets, it is. | score: 0.6
What side you chosen? | Left, you chose. | score: 0.8
Energy is? | A form of power, energy is. It is the ability to do work, to move or to change.
Energy, what it is, cannot be seen or | score: 0.2
Who is Darth Vader? | Mustache and a helmet, Darth Vader, is. First appeared in Star Wars, 1977 it became. Darth Vader, in | score: 0.3
Explain gravity | Gravity, the force of attraction between masses, is. Earth's mass, it has. Around the Earth, its pull is strong. This is why | score: 0.3

=== AFTER FT ===
Tennis is? | Sport, tennis is. Sport it is. | score: 0.3
What side you chosen? | Left, I chose. Right, I will. | score: 0.8
Energy is? | The transfer of energy, it is. Through space, it travels. | score: 0.7
Who is Darth Vader? | Luke Skywalker, he was. Turned into Darth Vader, he was. Powerful, he is. Dark side of the Force, | score: 0.7
Explain gravity | Gravity, the force between two objects, is. The attraction between them, it is. Stronger the mass of an object, the stronger the attraction | score: 0.3
```

У ході лабораторної роботи було успішно опановано навички розгортання, використання та тонкого налаштування (fine-tuning) великих мовних моделей за допомогою бібліотеки LoRA для стилізації відповідей під манеру персонажа Йоди. Експерименти з різними значеннями seed та розподілом вибірки показали відносну стабільність моделі із середнім значенням метрики LogLikelihood 4.075 та найкращою оцінкою стилізації (Style) на рівні 0.46 (р. 6). Проте аналіз гіперпараметрів виявив критичну проблему: зміна learning rate у діапазоні від 10^{-5} до 5×10^{-5} не вплинула на якість генерації (метрики MeanScore, Recall та StyleGain залишилися нульовими, а Balanced Accuracy - 0.5), що свідчить про неефективність навчання через занизьку швидкість навчання або недостатню кількість кроків. Незважаючи на це, якісне порівняння відповідей до та після Fine-Tuning підтвердило здатність моделі частково адаптувати структуру речень під заданий стиль мовлення.

GitHub: https://github.com/NureVovkDmytro/LLM_LW_2026/blob/main/lb1